

Module 12

Python OOP & Design Patterns

Abstract classes, Factory, Strategy, Singleton, and closures

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. OOP Refresher — Classes and Inheritance

```
class Animal:
    def __init__(self, name: str):
        self.name = name    # instance attribute

    def speak(self) -> str:
        raise NotImplementedError    # subclass must implement

class Dog(Animal):
    def speak(self) -> str:
        return f"{self.name} says: Woof!"

class Cat(Animal):
    def speak(self) -> str:
        return f"{self.name} says: Meow!"

dog = Dog("Rex")
print(dog.speak())    # "Rex says: Woof!"

# Polymorphism: same method, different behaviour
animals = [Dog("Rex"), Cat("Whiskers"), Dog("Buddy")]
for animal in animals:
    print(animal.speak())    # each calls its own speak()
```

2. Abstract Base Classes (ABC)

Python's ABC module lets you define **interfaces**: classes that specify what methods subclasses **MUST** implement, but don't implement them themselves. If a subclass doesn't implement an abstract method, Python raises a `TypeError` when you try to instantiate it.

```
from abc import ABC, abstractmethod

class EmbedClient(ABC):
    """Every embedding provider must implement these methods."""

    @abstractmethod
    def embed(self, texts: list[str]) -> list[list[float]]:
        """Embed a batch of texts. Never raises – returns empty lists on failure."""
        ...

    def embed_one(self, text: str) -> list[float]:
        """Convenience wrapper – implemented once, works for all subclasses."""
        results = self.embed([text])
        return results[0] if results else []

# Subclasses must implement embed():
class GeminiEmbedder(EmbedClient):
    def embed(self, texts: list[str]) -> list[list[float]]:
        # ... call Google Gemini API ...

class LocalEmbedder(EmbedClient):
    def embed(self, texts: list[str]) -> list[list[float]]:
        # ... run local fastembed model ...
```

```
# This raises TypeError – abstract method not implemented:
class BrokenEmbedder(EmbedClient):
    pass

e = BrokenEmbedder() # TypeError!
```

Note: ContentAutomation2's `EmbedClient` (`backend/app/agents/embedding/client.py:31`) is exactly this pattern — `GeminiEmbedder`, `LocalEmbedder`, `FallbackEmbedder`, and `NoOpEmbedder` all implement the `EmbedClient` interface.

3. Factory Pattern — Choose Which Implementation at Runtime

A factory is a function that decides which concrete class to create based on configuration or environment. The caller doesn't need to know about the concrete classes — it just asks the factory for an object that satisfies the interface.

```
from abc import ABC, abstractmethod

class EmbedClient(ABC):
    @abstractmethod
    def embed(self, texts: list[str]) -> list[list[float]]: ...

class GeminiEmbedder(EmbedClient): ...
class LocalEmbedder(EmbedClient): ...
class NoOpEmbedder(EmbedClient): ...

# ■■■ FACTORY FUNCTION ■■■
def make_embed_client(google_api_key: str | None, local_model: str) -> EmbedClient:
    """Returns the best available embedder based on configuration."""
    local = None
    try:
        local = LocalEmbedder(local_model)
    except ImportError:
        pass

    if google_api_key:
        gemini = GeminiEmbedder(google_api_key)
        if local:
            return FallbackEmbedder(primary=gemini, fallback=local) # see Strategy
        return gemini

    return local if local else NoOpEmbedder()

# Caller doesn't know OR care which class it gets:
client = make_embed_client(google_api_key="...", local_model="bge-base")
vectors = client.embed(["Hello"]) # works regardless of which backend
```

4. Strategy Pattern — Swap Algorithms at Runtime

The Strategy pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. The caller selects the strategy without knowing the implementation details.

```

class FallbackEmbedder(EmbedClient):
    """Strategy: try primary first, fall back to secondary on failure."""

    def __init__(self, primary: EmbedClient, fallback: EmbedClient) -> None:
        self._primary = primary
        self._fallback = fallback

    def embed(self, texts: list[str]) -> list[list[float]]:
        try:
            results = self._primary.embed(texts)
            # If ANY vector is empty, treat as full failure
            if any(not v for v in results):
                raise ValueError("Primary returned empty vectors")
            return results
        except Exception:
            return self._fallback.embed(texts)    # fall back to local

# Usage:
gemini = GeminiEmbedder(api_key="...")
local = LocalEmbedder("bge-base")

# Strategy 1: Gemini only
fast = FallbackEmbedder(primary=gemini, fallback=NoOpEmbedder())

# Strategy 2: Gemini with local fallback (resilient)
resilient = FallbackEmbedder(primary=gemini, fallback=local)

```

5. Singleton Pattern — One Instance for the Whole Process

A singleton ensures a class has only one instance. In ContentAutomation2, the local embedding model is expensive to load (~430 MB download + model init). The module-level cache is a simple singleton.

```

# Module-level cache (simplest Python singleton)
_local_model_cache: dict[str, object] = {}

class LocalEmbedder:
    def __init__(self, model_name: str):
        self._model = self._load(model_name)

    @staticmethod
    def _load(model_name: str):
        if model_name not in _local_model_cache:
            from fastembed import TextEmbedding
            _local_model_cache[model_name] = TextEmbedding(model_name)
        return _local_model_cache[model_name]    # same object every time

# First call: loads the model (slow, ~5 seconds)
e1 = LocalEmbedder("bge-base")
# Second call in same process: reuses cached model (instant)
e2 = LocalEmbedder("bge-base")
assert e1._model is e2._model    # True — same object

```

6. Closures — Functions That Capture Variables

A closure is a function that remembers the variables from the scope where it was defined — even after that scope has exited. This is how `make_tools()` works: each `@tool` function captures `supabase` and `arq_pool` from the outer scope.

```
def make_multiplier(factor: int):
    """Returns a function that multiplies its argument by factor."""
    def multiply(x: int) -> int:
        return x * factor    # 'factor' is captured from outer scope

    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5))    # 10
print(triple(5))    # 15
# 'factor' still accessible even though make_multiplier() finished

# Real usage in ContentAutomation2:
def make_tools(supabase: Client, arq_pool) -> list:

    @tool
    async def get_ideas(status: str = "pending_approval") -> str:
        # 'supabase' is captured from make_tools() scope – this is a closure!
        resp = supabase.table("ideas").eq("approval_status", status).execute()
        return str(resp.data)

    return [get_ideas]

# Call make_tools once at startup
tools = make_tools(db_client, pool)
# get_ideas now permanently has access to db_client and pool
```

7. dataclasses — Lightweight Data Containers

```
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class FetchResult:
    url: str
    title: str
    full_text: str
    word_count: int = 0
    paywall_detected: bool = False
    publication_date: Optional[str] = None

# Equivalent to a class with __init__, __repr__, and __eq__
result = FetchResult(url="https://...", title="RBI Rate Hike", full_text="...")
print(result)    # FetchResult(url=..., title=..., ...)

# When to use dataclass vs Pydantic:
# - dataclass: simple data containers, no serialisation needed, pure Python
```

- Pydantic: API responses, DB models, anywhere you need validation + JSON

8. Putting It All Together — The Embedding System

ContentAutomation2's embedding system uses ALL of these patterns:

- ABC (EmbedClient) — defines the interface every embedder must satisfy
- Factory (make_embed_client) — chooses the right implementation at runtime
- Strategy (FallbackEmbedder) — primary + fallback algorithm selection
- Singleton (_local_model_cache) — load the model once, reuse everywhere

Each pattern solves a specific problem. Together they produce a system that is flexible (swap backends), resilient (fallback on failure), and efficient (no redundant model loading).

9. Practice Exercises

- Exercise 1: Define an abstract Notifier class with an abstract method `send(message: str) -> bool`.
- Exercise 2: Implement `SlackNotifier` and `EmailNotifier` as concrete classes.
- Exercise 3: Write a `make_notifier(channel: str)` factory that returns the right notifier.
- Exercise 4: Implement `FallbackNotifier(primary, fallback)` using the Strategy pattern.
- Exercise 5: Add a module-level cache so `NotifierFactory` only creates one instance per channel.
- Exercise 6: Write a `make_tools(notifier)` closure factory where each tool closes over the notifier.
- Exercise 7: Chain them: Factory → Strategy → Singleton → Closure. Show the full working example.